Required Documentation Changes

Criterion A, B and C currently describe React + Flask + PostgreSQL. The application was actually built with Angular + FastAPI + SQLite (per your instruction), and several corrections were needed in the ERD, algorithm and validation flowcharts to make the design actually satisfy its own success criteria and test cases. This document lists every change your write-up needs so the documentation matches the working system.

1. Criterion A — Problem Specification

1.1 Computational Context — tech stack bullets

Original documentation	Required change	Reason
“For the GUI React.js and Bootstrap will be used.”	Replace with: “For the GUI Angular and Angular Material will be used.” Keep the same justification (SPA behaviour without full page reloads, pre-built components) — it applies equally to Angular Material.	Frontend framework changed from React to Angular for this build.
“I will be using flask as the web framework… I choose it over Django as it's lightweight and minimal…”	Replace with: “I will be using FastAPI as the web framework. I choose it over Flask/Django because it is lightweight like Flask, but adds native async support, automatic request validation via Pydantic, and auto-generated OpenAPI docs — useful when the frontend is a separate Angular SPA calling a typed JSON API.”	Backend framework changed from Flask to FastAPI for this build.
“PostgreSQL for the database as it handles multi-user access, strong data integrity through ACID transactions and has role-based access control so only authorised users can access the sensitive financial data”	Replace with: “SQLite for the database as it needs no separate server process, is fully file-based (simple to back up/ship with the app), and still gives ACID transactions for a single-server small-business deployment. Authorisation (which user may see which session) is enforced at the application layer instead of via database roles, since SQLite has no built-in role-based access control.”	Database changed from PostgreSQL to SQLite. SQLite genuinely has no role-based access control or robust multi-writer concurrency — the original justification no longer holds and must not be copied over unchanged.
“And libraries like pandas (data manipulation and reconciliation logic), pdfplumber (extract tabular data from the PDF files) and rapidfuzz (row and column matching) are necessary…”	Either (a) remove pandas from this sentence, keeping only pdfplumber and rapidfuzz, or (b) if you want to keep pandas for the mark scheme, explicitly use a pandas DataFrame somewhere in the matching step of the implementation so the documentation matches the code.	The working matching engine does not use pandas — rows are handled as plain Python objects for precise integer (paise) arithmetic. Documentation should not list a library that isn't actually used.

1.2 Success Criteria

Original documentation	Required change	Reason
“Tolerance: … difference up to 100 or percentage difference up to 3%”	Add a clarifying clause: “Percentage tolerance is always computed against the ledger amount (not the bank amount), and both absolute and percentage tolerances are inclusive at the boundary (a difference exactly equal to the tolerance still counts as a match).”	This rule is required for the matching algorithm to behave deterministically, but was never stated as a success criterion, only implied later in supporting material.
“Performance: … process and reconcile up to 500 transactions in 10-20 seconds”	Add: “If either uploaded file exceeds a configurable row limit (1000 rows), the app rejects the upload with an error message instead of attempting to process it.”	The Testing Strategy already expects an error for a 10,000-row file, but no success criterion or flowchart ever defines the cut-off that causes that error.

2. Criterion B — Planning

2.1 Decomposition

Original documentation	Required change	Reason
1. User Authentication and Security – “Hash passwords using werkzeug.security before storing in the database”	Replace with: “Hash passwords using bcrypt directly before storing in the database.”	werkzeug is a Flask dependency; FastAPI has no equivalent built in, and bcrypt can be called directly without an extra abstraction layer.
1. User Authentication and Security – “Set a secret key in Flask to cryptographically sign session cookies”	Replace with: “Sign a JWT (containing the user id and an expiry) with a secret key, and store it in an httpOnly cookie set by the API on login.”	FastAPI does not have Flask's built-in server-side session/cookie-signing mechanism; authentication state is carried in a signed JWT instead.
2. File Upload and Validation – “Make a file upload component in React” / “Define the POST route in Flask”	Replace “React” with “Angular” and “Flask” with “FastAPI” throughout this section.	Tech stack change.
3. Data extraction and Structuring – “Convert the extracted data into pandas dataframes for reconciliation”	Replace with: “Convert the extracted data into a list of plain Python row objects for reconciliation.”	Matches the actual implementation (no pandas dependency in the matching engine).
4. Transaction matching and Tolerance handling – “Make the matching logic in pandas to compare rows from the two dataframes”	Replace with: “For each ledger row, collect every bank row that passes the date and amount tolerance, score each candidate's description similarity with rapidfuzz, and assign the ledger row to the single best-scoring candidate (removing it from the pool so it cannot be reused by another ledger row).”	The original one-line description (“compare rows”) allows a naive first-match implementation, which fails on same-day/same-amount ambiguous pairs (see Criterion C, Section 3.3 of this document).
6/7. “Create a tolerance input in react” / “Build upload, reconcile, export, history buttons and pages using react”	Replace “react” with “Angular”.	Tech stack change.
7. “Display the reconciliation table with colour coded rows (green, yellow, red) using Bootstrap”	Replace “Bootstrap” with “Angular Material”.	Tech stack change.
7. “Display a progress bar in react while Flask is processing the PDFs”	Replace with: “Display a progress bar in Angular that polls a FastAPI endpoint for percentage-complete while the PDFs are processed in a background task.”	Tech stack change, and clarifies the polling mechanism (no server push was implemented).

2.2 Planning table (Week 1-2 row)

Original documentation	Required change	Reason
“Setup Flask, PostgreSQL, SQLAlchemy and React”	Replace with: “Setup FastAPI, SQLite, SQLAlchemy and Angular”	Tech stack change; SQLAlchemy is unchanged as it works identically over SQLite.

3. Criterion C — System Overview

3.1 ERD (Image 3)

Original documentation	Required change	Reason
MATCH_RESULT.ledger_txn_id (FK) and bank_txn_id (FK) shown as ordinary, implicitly required foreign keys	Make both columns nullable, and add a table-level rule: at least one of ledger_txn_id / bank_txn_id must be present.	Rows describing a transaction that is “Missing from bank” or “Missing from ledger” only have one side populated — the ERD as drawn cannot represent them.
MATCH_RESULT has columns: id, session_id, ledger_txn_id, bank_txn_id, match_status, reason, tolerance_applied	Add three more columns: date_diff_days (integer), amount_diff (numeric), similarity_score (numeric, 0-100).	The reconciliation report shows the day difference, amount difference and description similarity % for every row — without storing these, the report/export cannot be regenerated from the database.
TRANSACTION.amount is typed as float	Change to an integer column storing the amount in paise (minor currency units), e.g. ■250.00 stored as 25000.	Binary floating point cannot exactly represent amounts like 0.01 or 0.10, which risks a difference that should be exactly on a tolerance boundary being read as very slightly over or under it. Integer paise avoids this entirely.
RECONCILIATION_SESSION has columns: id, user_id, ledger_filename, bank_filename, date_tolerance_days, amount_tolerance, tolerance_type, status, created_at	Add: progress_pct (integer 0-100), error_message (string, nullable), description_threshold (numeric, default 80).	The progress bar (sequence diagram / SC “User interface and feedback”) needs somewhere to read percentage-complete from, error handling needs somewhere to store *why* a session failed, and the description-match threshold is a user-configurable value that isn't captured anywhere in the current schema.

3.2 Sequence diagram

Original documentation	Required change	Reason
Lane labelled “Flask Server”	Rename to “FastAPI Server”.	Backend framework change.
“Start confirmed” step implies a session-cookie is set by Flask's built-in session handling	Clarify: the server returns a signed JWT which the browser stores as an httpOnly cookie; this cookie is sent automatically on the later “Check progress” and “Get final results” calls.	FastAPI has no built-in session store like Flask — the diagram should show the actual auth mechanism used.

3.3 Transaction Matching Flowchart

The current flowchart iterates ledger rows one at a time and, for each one, validates date then amount then fuzzy-matches description against what looks like a single bank row, marking it MATCHED or Partial and moving straight to the next ledger row. This does not work once more than one bank row can pass the date+amount check for the same ledger row.

Original documentation	Required change	Reason
Steps 4-6 (“Validate Date” → “Validate Amount” → “Fuzzy Matching”) operate on one candidate bank row per ledger row, first-match wins	Replace with: for each ledger row, first collect every bank row (not yet used by an earlier ledger row) that passes both the date and amount tolerance into a candidate list. If the list is empty, mark the ledger row “Missing from bank”. Otherwise, fuzzy-match the description against every candidate and keep only the single highest-scoring one; mark that bank row as used so no other ledger row can claim it. Then apply the existing ≥80%/<80% split for Matched vs Partial.	With a naive first-match approach, two ledger rows and two bank rows that share the same date and amount (e.g. two advance payments made on the same day for the same amount to different suppliers) can be paired with the wrong counterpart, because the algorithm never compares descriptions across all candidates before committing to a match.

Original documentation	Required change	Reason
No explicit statement that a bank row, once matched, is removed from the pool of candidates	Add a note/step: “a bank row that has been assigned to a ledger row (whether Matched or Partial) is removed from the candidate pool and cannot be assigned to any other ledger row.”	Without this rule the same bank transaction could satisfy two different ledger entries, which is not one-to-one reconciliation.

3.4 File Upload Validation Flowchart

Original documentation	Required change	Reason
No step checks the uploaded file's size before extraction	Insert a check immediately after “Check: Extension is .pdf?”: “Check: file size > 0 bytes?” → No: “Error: empty file”.	The Testing Strategy has a dedicated “Upload empty 0-byte file” test case expecting an error message, but no flowchart step produces that error.
No step limits how many rows can be extracted before matching begins	Insert a check after “Extract tables”: “Check: row count ≤ maximum supported (e.g. 1000)?” → No: “Error: too many rows”.	The Testing Strategy expects a 10,000-row file to produce an error, but Success Criteria only promises performance up to 500 rows — the flowchart needs an explicit cut-off that causes the rejection.
“Check: Tables extracted?” → No → “Error: no tables or corrupt PDF” is the only handling for unreadable PDFs	Add a separate, earlier check right after opening the file: “Can the PDF be opened without a password?” → No: “Error: password-protected or corrupted PDF”.	Opening an encrypted PDF raises a different kind of error than “no tables found” and should be caught explicitly — the Testing Strategy has its own row for password-protected PDFs.

3.5 Pseudocode

Original documentation	Required change	Reason
“hashed = werkzeug.generate_password_hash(password)”	Replace with: “hashed = bcrypt.hashpw(password, bcrypt.gensalt())”	werkzeug is Flask-specific; bcrypt is called directly instead.
“SET session cookie with user.id” / “RETURN redirect to /reconcile”	Replace with: “token = create_jwt({'sub': user.id}, secret_key, expiry)” and “SET httpOnly cookie with token; RETURN user profile as JSON”.	FastAPI serves a JSON API for the Angular SPA to call, rather than performing a server-rendered redirect; the SPA's router (not the server) decides which page to show next.

3.6 Testing Strategy — rows to add

Original documentation	Required change	Reason
(not currently present)	Automatic Matching / Normal: two ledger rows and two bank rows share the same date and amount but different descriptions → expected result: each ledger row pairs with the bank row that has the closer-matching description, not simply the first candidate found.	Directly tests the corrected matching algorithm in Section 3.3 above.
(not currently present)	Security / Abnormal: a logged-in user requests another user's session id directly by URL → expected result: request is rejected (not found), not returned.	Confirms sessions are private per-user, which the current Security row (login redirect only) does not cover.
(not currently present)	Tolerance / Normal: percentage tolerance is confirmed to be calculated from the ledger amount, not the bank amount.	The percentage-tolerance rule is used by the matching algorithm but was never written down as something to test.

3.7 GUI screens

Original documentation	Required change	Reason
Screens use Bootstrap-style components in the wireframe captions	Update captions/labels to Angular Material equivalents (e.g. “mat-card”, “mat-table”, “mat-progress-bar”) instead of Bootstrap component names.	Tech stack change.
Nav bar mockup on Screen 2 includes a “History” link, but no History screen is designed anywhere in Criterion C	Add a 4th wireframe: “Screen 4 – History” showing a list of past reconciliation sessions (date, file names, tolerance used, summary counts) with an action to re-open each one.	Closes a gap between the navigation design and the screens that were actually specified.

Full technical detail (revised SQL schema, corrected matching pseudocode, REST API contract) is in Docs/DEVELOPMENT_PLAN.md, which was used as the basis for the implementation that these documentation changes are being aligned to.